Team Project
2026 Spring, SWPP

All scheduled dates are tentative.

Goal

In this project, you'll work as a team with other students to write a simple compiler that reads an LLVM IR program and emits an optimized assembly for an imaginary machine.

Each team should

- Use LLVM **22.1.0** built with the script in class repo for your project.
- Use GitHub to collaborate and code-review teammates' implementation.

At the end of the semester, we'll have a competition to evaluate the performance of each team's compiler.

Schedule (subject to change)

Date (Tentative)	Details
4/16	Introduce your team in class
4/17 - 4/23	Sprint 0 - Read codes, setup development envs and CI, etc.
4/23	Submit two documents: - Requirement and specification - Planning
4/24 - 5/7	Sprint 1 - ~10 days (From Saturday): development - 3+ days (until Thursday): code review+revision - Last day: progress report & req/spec plans
5/8 - 5/21	Sprint 2
5/22 - 6/4	Sprint 3
6/5 - 6/8	Wrap-up
6/9	Competition

<u>(1) Sprint 0</u>

Introduce your team in the class

Each team will have about 5 minutes to introduce themselves in the class.

- Introduce your team & teammates.
- Explain what you want to learn during the team project about software engineering.
- Suggest 5+ optimizations (either simple or complex) with example codes.
- Prepare short presentation slides.
- The talk is lightweight; your talk does not need to get very technical.

Pre-project Documentation

You should submit two documents (A. requirement and specification, B. planning) before the first sprint. You should submit the document to TAs via eTL. Each team is assigned an eTL group, so anyone can submit the document on behalf of their team. Each document should not be more than 7 pages. You may use Korean or English for these documents. The document should be in ***pdf*** format.

A. Planning
    - File name should be “sprint0-teamN-planning.pdf” (N is your team number)

    - Briefly explain optimizations that your team would like to implement during the 3 sprints.
    - It should include each member's plan for 3 sprints.
    - Optimizations can be implemented with your teammates.
    - Optimizations can be implemented through multiple sprints.
B. Requirement and specification
    - File name should be "sprint**1**-teamN-reqspec.pdf" (note the sprint number)
    - Describe the optimizations that you are going to implement in sprint 1. For each optimization, describe the following information.
        i. Description
        ii. An algorithm in a high-level pseudo-code
        iii. At least 3 pairs of IR programs, before and after the suggested optimization

Preparing the Project Skeleton

Once the team project repository template has been released, each team should create their own team repository from the template provided. During sprint 0, students can directly push commits that contain project skeleton code, CMakeLists configuration, or GitHub Actions configuration into the team repository. These commits are exempt from PR criteria (described below) and grading.

Especially, automated testing on the GitHub Actions should be fully functional by the beginning of sprint 1. Details about testing are described later in this document.

<u>(2) Sprint 1/2/3</u>

Each sprint consists of

- **Development phase** (Up to 10 days, starting from Friday)
    - Implement the planned features
    - Send pull requests to the main repository
    - Assign reviewers.
- **Code-review phase** (At least 3 days, until Wednesday)
    - Assigned reviewers review the pull requests
    - Reviewee updates the pull request according to the comments.
    - If the update is complete, the pull request is merged into the main repository
- **Documentation phase** (On Thursday)
    - Write a progress report for this sprint, and requirements and specifications for the next sprint. Submit them via eTL.

You may use Korean or English in each process.

Pull Requests

- A pull request contains an implementation of a single feature.
- The title of a pull request should start with [Sprint N]. (N is the current sprint number)
- It should contain unit tests to check the added functionality.
- Its line diff should be at most around 500 lines except comments, spaces, tests, and non-C++ code files such as .gitignore or CMakeLists.txt.
    - Use a formatter whenever you write code. It's also helpful to use Git hooks or similar tools to automatically format your code before submitting a PR. You may use any formatter, but we recommend using the one we provided in the `.clang-format` file.
    - We'll count the LOC after running the formatter on your code. Formatting rules can be found in included `.clang-format`
- It should satisfy the good pull request conditions that are described in Apr.9th's practice session
- You may merge to any branch on the repository. However, only the PRs that are merged into the **main** branch will be graded.
    - If the opt. pass PR is to be merged to the main branch, it must be reviewed.
    - Otherwise, reviews are optional but recommended.
- Each pull request should be merged using 'squash and merge' by the pull request writer (not reviewers!)
- Detailed information about PR grading is provided at the end of this document.

Policy for Writing Pull Requests & Reviews

- Each student can merge up to 6 pull requests per sprint.
    - If your team has less than four teammates for any reason, you may take the 'merge chance' of the missing teammate(s) as well.
- Each student should merge at least one pull request per sprint.
    - Only 'graded' PRs count; otherwise, we cannot grade your performance on a sprint and have to give you zero points.
    - We will count co-authored PR as one PR per each person.
- For each pull request, 2 or more reviewers should be assigned.
- Every PR must pass the CI at the time of merge.
    - Merging the PR that fails the CI will be penalized, with no exception.
- If necessary, a team can write pull requests that make ***non-functional changes*** such as directory structure changes / source file splitting / etc.
    - Please add "[Sprint N, NFC]" at the beginning of the title.
    - This pull request should be reviewed but not as much as functional ones.
    - This pull request is exempt from the 6 pull request restriction.
    - We'll be generous about what counts as 'non-functional':
      As long as your PR does not change the function of the compiler code, you may call it NFC.
- If necessary, a team can make reverting pull requests that ***revert previous PR***

    - Please add “[Sprint N, Revert]” at the beginning of the title.
    - Please add “[Sprint N, Reverted]” at the title of the reverted PR.
    - This pull request should be reviewed but not as much as functional ones.
    - ‘Revert’ and ‘Reverted’ pull requests are exempt from the 6 pull request restriction.
    - While ‘Revert’ PR will not be graded, we will still use ‘Reverted’ PR for grading.

- ***TAs’ codebase update*** should be applied in the form of PR as well.
    - Please add “[Sprint N, Update]” at the beginning of the title.
    - This pull request does not have to be reviewed, but it should still pass the CI!
    - You may apply Update PRs at any moment of the project period, but all updates must be applied to the team repository by the end of wrap-up period, as they may contain critical bug fixes.

Existing Optimizations

LLVM has many optimization passes, and simply calling these functions from your project can sometimes bring large performance improvements.

Each existing optimization should be added via separate PR, and should come with unit tests as well. This does count in six PR limits. Adding an existing pass as NFC will be penalized.

Existing Analyses

LLVM has many analysis passes as well.

Unlike optimization passes (that actually modify the code structure), you may include and use the analysis passes as part of your pass implementation. Only the pass that you have implemented will count in six PR limits.

Writing & Running Unit Tests for Optimization Pass

- Each optimization pass PR should contain at least 3 unit tests.
- Unit tests are responsible for checking the functionality of ‘single’ optimization pass.
    - Testing the output of `opt --<new-opt-pass>` is therefore recommended
- Each unit test should be an LLVM IR program with FileCheck comments.
- Running `ctest --test-dir build` should run all added unit tests.
- Each commit and pull request to the team repository should trigger GitHub Actions and run these tests.
- Adding Alive2 tests is not mandatory but is highly recommended.

Writing & Running Unit Tests for Analysis Pass

- Each analysis pass PR should contain at least 3 unit tests.
- Unlike optimization passes, analysis results are not visible in IR programs, and requires direct access into LLVM’s internal data structure to check its functionality.
- Therefore, each unit test should be a C++ code that loads an LLVM IR program, runs the program via analysis pass, and asserts that your pass functions as expected.

- Other than the implementation method, the rule for optimization pass unit tests apply the same.

Main Repository

- The main repository should contain a **main** branch.
- After each commit, the project should work correctly. TA will check

```
./compile.sh # we will modify directory variables as necessary
ctest --test-dir build
```

  from the root directory of the branch.

Using GitHub Issue

- Please use the GitHub issue to show that you are actively communicating with people.
- If there was an offline discussion and it wasn't written as comments at Pull Request, please record it at GitHub Issue.

Program-Specific Optimization

- Optimizing a program based on function name matching is not allowed.
- Replacing an entire program implementation with pre-written code is not allowed.
- Your optimization should modify the code based on logical analysis, not via syntax matching.

End-of-the-Sprint Documentation

At the end of each sprint, each team should submit the progress report and the updated requirements/specifications for the next sprint. The document should be in ***pdf*** format.

A. Progress report
   - File name should be "sprintN-teamM-progress.pdf" (N is the **current** sprint)
   - Describe each member's progress at the end of the sprint.
   - If you did not finish what you have planned, explain why.
   - Include the results of applying your passes to all benchmarks (distributed at the beginning of the project) as well as your own tests to show the effectiveness of your optimizations. Examples may include performance enhancements, applicability (how many cases do your passes have effects), etc.
   - If there is an update in planning, please submit the updated part as well.
   - This document should not exceed 5 pages.

B. Requirement and specification
   - File name should be "sprintN-teamM-reqspec.pdf" (N is the **upcoming** sprint)
   - Describe optimizations that you are going to implement in the next sprint. For each optimization, description / pseudo-code / IR programs should be included.
   - This document should not exceed 7 pages.

You should submit two documents on eTL. You may use Korean or English.

	~ Sprint 0	~ Sprint 1	~ Sprint 2	~ Sprint 3
	4/23 11:59 pm	5/7 11:59 pm	5/21 11:59 pm	6/4 11:59 pm
Planning	O			
Requirement and specification	O (1)	O (2)	O (3)	
Progress report		O (1)	O (2)	O (3)

(3) Wrap-up & Competition

After 3 sprints, you will be given a few days to wrap up your implementation.
In this period, you may commit codes as much as you want (no code review, no line diff constraint, …)

On the last day, we'll run a competition, estimating the correctness & efficiency of the compiler.

AI Policy

The use of AI is allowed for code, pull requests, review comments, and project documents. However, students are fully responsible for the correctness and quality of anything they submit. Before submission, you must carefully check whether the AI-generated content is accurate, technically sound, and appropriate for the project. Submissions that contain incorrect, misleading, or unnatural AI-generated content may receive a substantial penalty.

If AI is used in a pull request, review, or document, this must be explicitly disclosed.

In particular, students must not modify code based only on function names or superficial pattern matching. Code should be changed only when the student actually understands its behavior and purpose.

Sprint PR Grading Policy

This project has three sprints. For each sprint, the score is composed of:

- **Personal score: 20 points**
- **Team score: 5 points**

So, each sprint is worth 25 points.

Personal Score (20 points)

The personal score for a sprint is based on the average score of that student's PRs in the sprint.

- Minimum PR Requirement
    - A student must submit at least 2 PRs in the sprint to be eligible for the full 20-point individual score.
    - If a student submits only 1 PR, the individual score is capped at 10 points, even if the PR itself receives a higher score.
    - If a student submits 0 PRs, the individual score is 0 points.

Grading PR

Each PR is graded using the following rubric:

- **Explanation: 10 points**
- **Performance Evaluation: 4 points**
- **Correctness: 8+1 points**

The raw maximum for a PR is therefore 23 points, but the recorded PR score is capped at 20 points. In other words, Recorded PR score = min(Raw PR score, 20).

PR Rubric

1) **Explanation (10 points)**
   This category evaluates whether the PR clearly explains the problem and the implementation.
   a) **Issue Description**
      - The PR should briefly explain what the current issue is.
      - **Full credit:** The PR clearly states what problem, inefficiency, or missed optimization opportunity exists.
      - **No credit:** The PR does not describe the issue at all, or only says that something was changed without explaining why.

   b) **Implementation / High-Level Algorithm**
      - The PR should explain how the pass is implemented.
      - **Full credit:** The PR describes the main implementation idea clearly. It includes a high-level algorithm description or equivalent explanation of the core transformation logic.
      - **Partial credit:** The PR explains the implementation only partially, or at a very superficial level.

- **Low credit:** The reviewer must infer the implementation only from the diff.

2) **Performance Evaluation (4 points)**
   This category evaluates whether the PR includes quantitative evidence and analysis. The amount of speedup itself is **not** part of the grading criteria. What matters is whether the PR includes **quantitative evidence** and a **reasonable analysis**.
   a) **Quantitative Result**
      - The PR should include numerical performance results.
      - **Full credit:** The PR clearly states what benchmark(s) or cases were tested and includes a table, figure, or clearly written numerical comparison.
      - **No credit:** The PR only claims that performance improved, with no numbers.

   b) **Analysis**
      - The PR should include some interpretation of the result.
      - **Full credit:** The PR explains what the result means, even briefly.
      - **No credit:** Numbers are given with no discussion.

3) **Correctness (8+1 points)**
   This category evaluates whether the PR provides evidence that the optimization is correct.

   a) **Unit Tests and Example Program Tests**
      - The PR must show that **both** unit tests and example program tests were run successfully.
      - The PR should explain what was tested.
      - **Full credit:** The PR clearly shows that both unit tests and example program tests passed. The PR states which tests were run, or gives a sufficiently clear summary of test coverage.
      - **Partial credit:** The PR says tests were run, but the scope is unclear.
      - **No credit:** No meaningful test report is provided.
      - **Important rule:** If either one is missing, this item receives 0 points.
   b) **Correctness Discussion**
      - The PR should provide additional evidence that the optimization is safe.
      - Examples of acceptable evidence:
        - discussion of safety conditions,
        - reasoning about edge cases,
        - explanation of why the transformation preserves correctness,
        - limitations intentionally left unoptimized.
      - **Full credit:** The PR gives a technically meaningful correctness discussion.

        - **Partial credit:** The PR includes only a weak or incomplete correctness argument.
        - **No credit:** No correctness discussion is provided.
    c) **Alive2 Bonus (Optional)**
        - Alive2 is **not required**, but it can earn a bonus point.
        - **Full credit:** The PR reports meaningful Alive2 usage and result(s).
        - **No credit:** Alive2 is not used, or is mentioned without any meaningful evidence.

Team Score

- Each team starts with 5 points per sprint. Points are deducted for process issues.
    - **Insufficient PR review:** -1 per PR
    - **CI did not pass:** -1 per PR
    - **PR was not squash-merged:** -1 per PR
    - **Direct push to main branch:** -1 per incident
- The team score cannot go below 0 points.

What counts as an insufficient PR review?

- A PR review is considered insufficient if it does not provide meaningful technical feedback.
- Examples of insufficient review:
    - “LGTM”
    - “Looks good”
    - “Good job”
    - emoji-only or approval-only comments
    - comments only about title, formatting, or trivial wording
- Examples of a sufficient review:
    - raising a correctness concern
    - asking about an edge case
    - questioning performance evaluation
    - asking whether tests are adequate
    - discussing algorithm design or implementation choices

General Notes

- A technically strong PR can still lose points if it is poorly explained.
- A well-written PR can still lose points if it lacks quantitative evidence or correctness validation.
- Claims without evidence will not receive full credit.
- The grading focuses on whether the PR communicates the work clearly and supports its claims with evidence.

Example PR

[Sprint 2] Strength reduction for multiplication by powers of two

Summary

This PR implements a simple strength-reduction optimization in our LLVM pass.
When the pass finds an integer multiplication by a constant power of two, it replaces the multiplication with a left shift by `log2(constant)`. For example, `x * 8` is transformed into `x << 3` when the transformation is valid.

Issue Description

In the current implementation, expressions such as `x * 2, x * 4`, and `x * 8` remain as integer multiplication instructions throughout our optimization pipeline. This is a missed optimization opportunity because multiplication by a power-of-two constant can often be expressed more simply as a shift.
Even if later backend stages may sometimes perform a similar simplification, it is still useful to canonicalize this pattern earlier in our pass pipeline. Doing so can reduce instruction cost, simplify IR, and expose additional opportunities for later optimizations.

Implementation / High-Level Algorithm

Main idea

The pass matches `mul` instructions where one operand is an integer constant and the other is an arbitrary integer value. If the constant is a positive power of two, the multiplication is replaced with an equivalent left shift.

High-level algorithm

1. Iterate through all instructions in each function.
2. Identify integer `mul` instructions.
3. Check whether one operand is a constant integer.
4. Check whether the constant is a positive power of two.
5. Compute the shift amount `k = log2(C)`.
6. Replace:
   - `mul x, C` with `shl x, k`
   - `mul C, x` with `shl x, k`
7. Leave all other cases unchanged.

Important implementation details

- The transformation is applied only to integer scalar types in this version.
- The pass only transforms cases where the constant operand is a positive power of two.
- Cases such as `x * 0, x * 1,` negative constants, non-power-of-two constants, and vector types are intentionally left unchanged in this PR.
- This keeps the transformation simple and avoids mixing this PR with other simplification rules.

Why this implementation is reasonable

A shift is a natural representation for multiplication by powers of two. Restricting the optimization to simple scalar integer cases makes the pass easier to validate and reduces the risk of introducing bugs in edge cases.

Performance Evaluation

Benchmarks

I evaluated the optimization on the provided example programs and several microbenchmarks containing repeated arithmetic patterns.

Quantitative Results

Benchmark	Baseline (ms)	Optimized (ms)	Change
`arith_loop`	121.4	113.6	6.4% faster
`image_kernel`	204.7	197.1	3.7% faster
`matrix_scalar`	88.9	85.8	3.5% faster
`vector_ops`	97.8	97.5	0.3% faster
`simple_micro_mul_pow2`	45.2	39.4	12.8% faster

Analysis

The optimization provides the clearest benefit on programs that contain many integer multiplications by compile-time powers of two that survive until our pass runs. This is why the improvement is most visible in `arith_loop` and `simple_micro_mul_pow2`.

The effect is smaller on programs such as `vector_ops`, where this pattern appears less often or is not on the hot path. In some cases, backend code generation may already perform a similar simplification, so the speedup is not always large. That is expected.

The goal of this PR is not to guarantee a large improvement on every benchmark, but to show that the optimization is quantitatively measurable and behaves as intended.

Correctness

Unit Tests

I added unit tests for the following cases:

- `x * 2 -> x << 1`
- `x * 4 -> x << 2`
- `x * 8 -> x << 3`
- `2 * x -> x << 1`
- non-power-of-two constants are not transformed
- `x * 0` is not handled by this PR
- `x * 1` is not handled by this PR
- negative constants are not transformed

- non-integer or unsupported cases remain unchanged

All added unit tests passed.

Example Program Tests

I ran the full example-program test suite after applying this optimization.

Test result summary:

- all provided example programs compiled successfully
- all example programs produced the same output as the baseline compiler
- no regression was observed in the existing example tests

So both the unit tests and the example program tests passed successfully.

Correctness Discussion

This transformation is safe in the restricted cases handled by this PR because multiplication by a positive power-of-two constant is semantically equivalent to a left shift by the corresponding amount for the same integer value representation used in this pass.

To keep the transformation conservative, this PR intentionally avoids broader cases. In particular:

- non-power-of-two constants are ignored
- negative constants are ignored
- unsupported types are ignored
- vector cases are left for future work

By restricting the optimization scope, the pass only rewrites cases that are easy to reason about and test.

Alive2

I also checked representative cases with Alive2.

Alive2 results:

- `mul i32 %x, 2 <-> shl i32 %x, 1`: verified
- `mul i32 %x, 8 <-> shl i32 %x, 3`: verified
- `mul i64 %x, 16 <-> shl i64 %x, 4`: verified

Alive2 was used only for representative scalar cases in this PR, but the results are consistent with the intended transformation rule.